✅ What is Terraform?

Terraform is an **Infrastructure as Code (IaC)** tool created by Mitchell Hashimoto and released by HashiCorp in 2014.

It allows you to **provision and manage infrastructure using code** instead of manual configuration.

Terraform uses **HCL (HashiCorp Configuration Language)** — it looks similar to JSON but is more human-readable.

✅ What is Infrastructure as Code (IaC)?

Infrastructure as Code is the practice of **managing and provisioning infrastructure using code instead of manual processes**.

Instead of:

- Logging into AWS
- Clicking buttons
- Creating EC2 manually

You define everything in a `.tf` file and Terraform creates it automatically.

✅ Types of Infrastructure as Code

1️⃣ Imperative IaC

- You define **how** to do something step by step.
- Example: Scripts (Shell, Ansible in some cases).
- You control execution flow.

2️⃣ Declarative IaC (Terraform uses this ✅)

- You define **what you want** (desired state).
- Terraform decides **how to achieve it**.

Example:

You write:

```
I need 1 EC2 instance
```

Terraform checks current state and creates only what is missing.

✅ Why Terraform is Powerful?

- Multi-cloud support (AWS, Azure, GCP)
- Declarative approach
- Maintains state
- Handles dependencies automatically
- Reusable modules

✅ Important Terraform Commands (Interview Ready)

🔹 terraform init

- Initializes the project
- Downloads required provider plugins
- Sets up backend
- Creates `.terraform` directory

🔹 terraform validate

- Checks syntax errors in configuration files

🔹 terraform plan

- Shows execution plan
- Displays what will be created, updated, or destroyed
- Does NOT make changes

🔹 terraform apply

- Executes the plan

- Creates/updates infrastructure

🔹 terraform destroy

- Deletes all resources defined in configuration

✅ What is a Terraform State File?

Terraform creates a file called:

```
terraform.tfstate
```

Purpose:

- Stores the **current state of infrastructure**
- Maps real AWS resources to your configuration
- Tracks resource IDs

Without state file, Terraform does not know:

- What already exists
- What needs to change

👉 Very important for interview.

✅ What Happens During `terraform init`?

When you run:

```
terraform init
```

Terraform:

1. Downloads provider plugins (e.g., AWS provider)
2. Initializes backend configuration
3. Creates `.terraform` folder
4. Prepares working directory

It does NOT create resources.

✅ What is Remote Backend?

By default, Terraform stores state locally.

Problem:

- Not safe for teams
- Risk of deletion
- No locking mechanism

Remote Backend means:

Storing state file in remote storage like:

- AWS S3
- Azure Storage
- Terraform Cloud

For AWS best practice:

- Store state in S3
- Use DynamoDB for state locking

This prevents multiple users from modifying infrastructure at the same time.

✅ How Terraform Handles Dependencies?

Terraform automatically builds a **dependency graph**.

Example:

If:

- EC2 depends on Security Group
- Security Group depends on VPC

Terraform automatically creates:

VPC → Security Group → EC2

This is called **implicit dependency**.

You can also define explicit dependency using:

```
depends_on = [aws_security_group.my_sg]
```